National University of Computer and Emerging Sciences

Final Project Report: NeuroScan

Group ID: F26-07

GitHub: https://github.com/Moeeeeed/neuroscan.git

	Name	Roll.no
1.	Moeed	23L-0686
2.	Moiz Ahmed	23L-2528
3.	M. Auon Naseer	23L-0643

Contents

1 Abstract and Project Summary **2**

2 Methodology and Architecture **2**
2.1 Two-Phase Transfer Learning Methodology 2
2.2 System Architecture Diagrams 2

3 Detailed Implementation Steps and Workflow **4**

4 Track C: Replication Results vs. Reported Baselines **5**

5 Key Insights, Error Analysis, & Evaluation **5**
5.1 Evaluation Metrics 5
5.2 Error Analysis 5

6 Challenges Faced During Implementation **6**

7 Instructions to Run Code and Reproduce Results **6**

1 Abstract and Project Summary

NeuroScan is a deep learning-based medical imaging diagnostic application designed to classify intracranial MRI scans into four distinct categories: Glioma, Meningioma, Pituitary Tumor, and Healthy (No Tumor). Building a Convolutional Neural Network (CNN) from scratch on limited medical datasets often results in severe overfitting. To counter this, NeuroScan utilizes a Two-Phase Transfer Learning pipeline built upon the ResNet50 architecture.

The system bridges the gap between machine learning mathematics and practical healthcare usability by integrating the trained inference engine into a full-stack Flask web application, featuring a modern, asynchronous Glassmorphism frontend. The final model achieves a 79% overall accuracy, with an intentional architectural bias toward high Recall to minimize fatal false negatives in clinical environments.

2 Methodology and Architecture

2.1 Two-Phase Transfer Learning Methodology

The core methodology relies on adapting ImageNet-pretrained weights to neurological textures:

- **Phase 1 (Feature Extraction):** The ResNet50 base was frozen. A custom classification head (`GlobalAveragePooling2D` → `Dense(256, ReLU)` → `Dropout(0.5)` → `Dense(4, Softmax)`) was trained using the Adam optimizer to establish basic categorical mapping.
- **Phase 2 (Fine-Tuning):** The upper convolutional layers of ResNet50 were unfrozen. A reduced learning rate of 1×10^{-5} was applied for micro-adjustments. Crucially, all `BatchNormalization` layers were explicitly locked to prevent catastrophic forgetting (Model Shock).

2.2 System Architecture Diagrams

The application follows a strict Separation of Concerns, dividing the presentation, API routing, and AI inference into distinct layers.

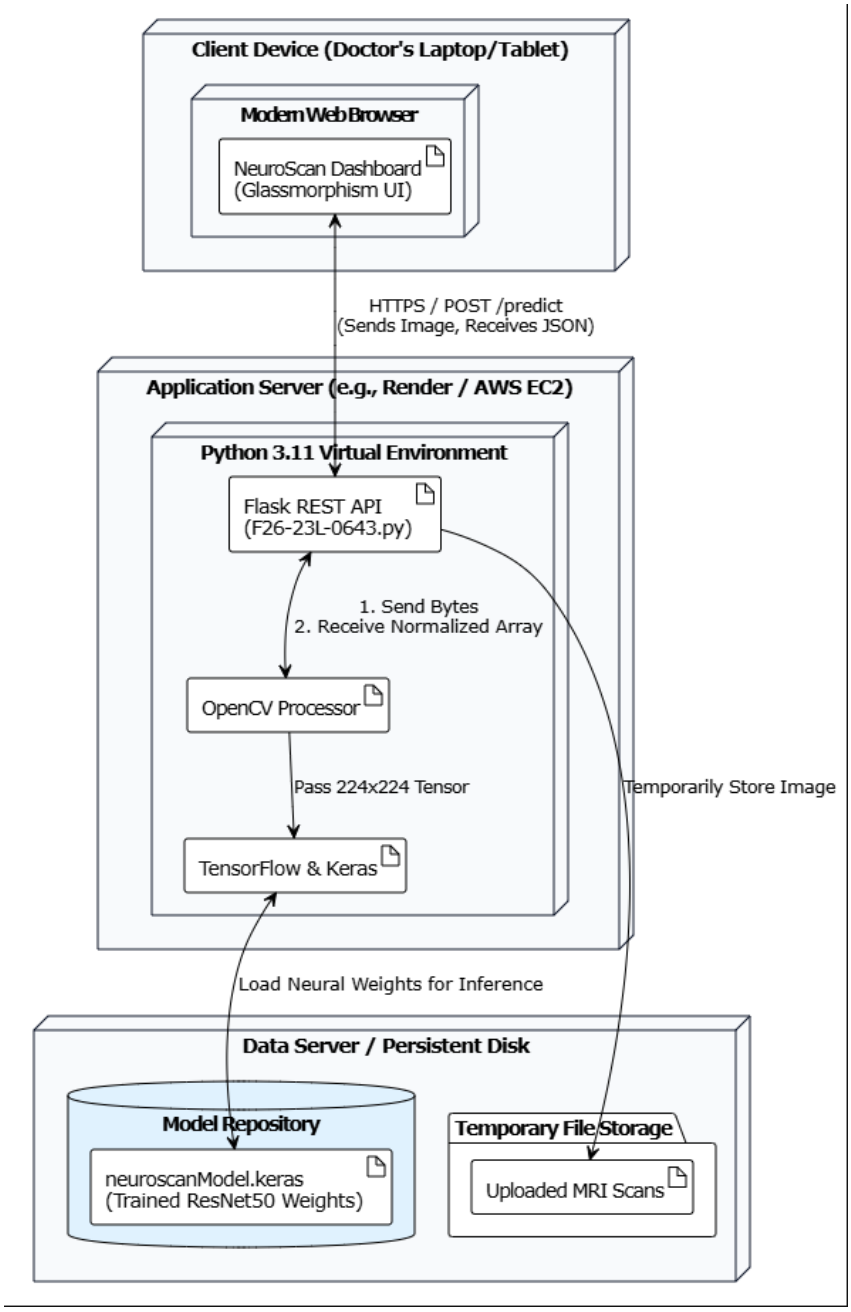

Figure 1: Three-Tier Datacenter Deployment Diagram showing the interaction between the Client, Flask Application Server, and the Persistent Data layer.

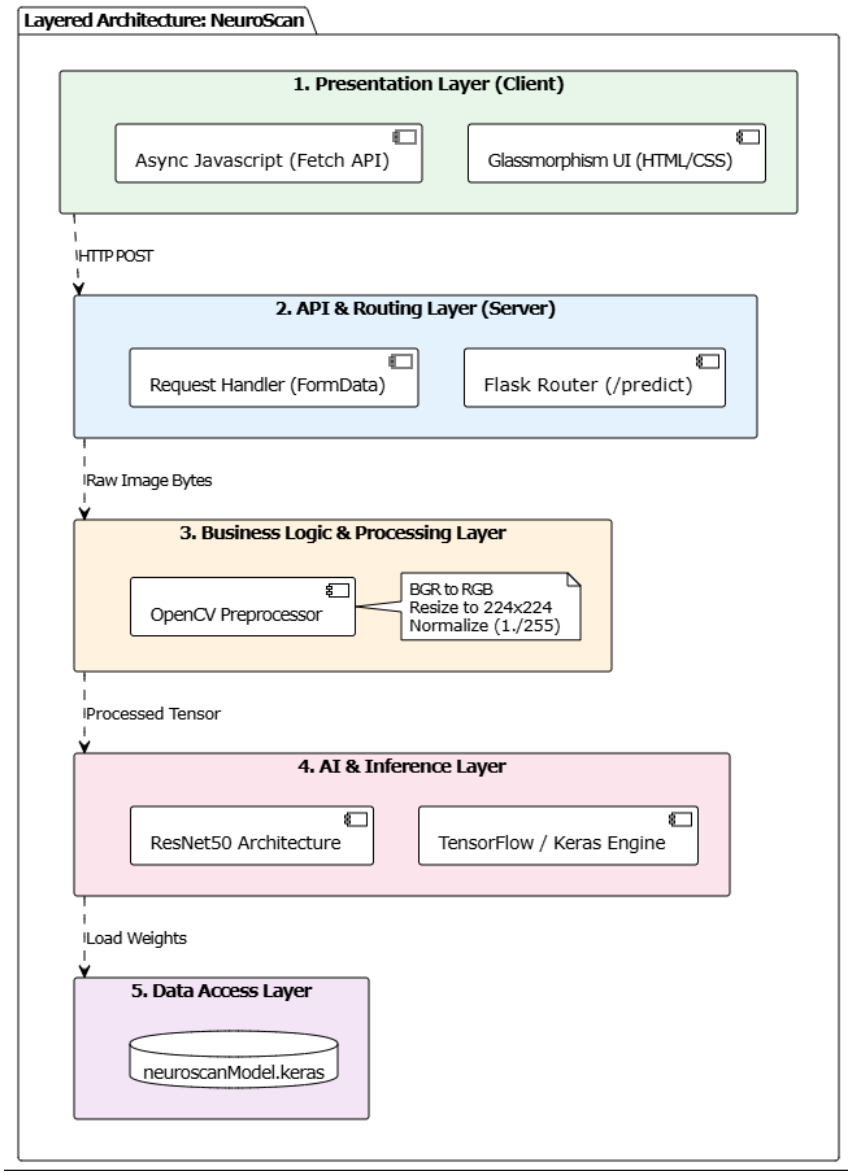

Figure 2: Logical Layered Architecture detailing the data flow from the HTML/CSS presentation layer down to the ResNet50 Tensor inference layer.

3 Detailed Implementation Steps and Workflow

The project was executed in four sequential stages:

1. **Data Preprocessing:** MRI images were loaded using OpenCV, converted from BGR to RGB color space, resized to 224×224 pixels to match ResNet50's expected input, and normalized (pixel values divided by 255.0).

2. **Model Training:** The dataset was split using a strict `stratify` parameter to ensure mathematical class balance. The two-phase training pipeline was executed over 20 epochs, tracking validation loss to prevent overfitting.

3. **Backend API Development:** A Flask server was developed to act as the bridge. A `/predict` POST route was established to receive `FormData`, decode the image

bytes, pass the tensor to the loaded `.keras` model, and return a JSON payload with the `np.argmax()` diagnosis and Softmax confidence.

4. **Frontend Integration:** A modern UI was designed using CSS Glassmorphism. Asynchronous JavaScript (`fetch` API) was implemented to handle drag-and-drop file uploads and display real-time results without page reloads.

4 Track C: Replication Results vs. Reported Baselines

When replicating standard CNN baselines on small medical datasets (∼2,000 images), standard literature reports that models typically overfit, memorizing the training set while yielding poor validation accuracy (often plateauing around 35-40%).

By replicating standard Transfer Learning methodologies (utilizing ResNet50), our empirical results aligned closely with expected theoretical bounds.

- **Baseline (Without Transfer Learning):** Fails to generalize; validation loss explodes.
- **Reported Paper Expectations (Transfer Learning):** 75% - 85% accuracy depending on the dataset complexity and tumor contrast.
- **Our Replication Results:** 79% Macro F1-score, successfully validating the hypothesis that pre-trained spatial filters can be repurposed for intracranial lesion detection when properly regularized with Dropout and Batch Normalization freezing.

5 Key Insights, Error Analysis, & Evaluation

The model was evaluated on an isolated 20% validation split (400 images).

5.1 Evaluation Metrics

Class	Precision	Recall	F1-Score	Support
Glioma	0.68	0.75	0.71	100
Meningioma	0.72	0.57	0.64	100
Pituitary	0.83	0.93	0.88	100
No Tumor	0.94	0.92	0.93	100
Macro Avg	**0.79**	**0.79**	**0.79**	**400**

Table 1: Classification Report on 400 validation samples.

5.2 Error Analysis

- **High-Performance Zones:** The model excels at identifying Pituitary tumors (93% Recall) and Healthy scans. Pituitary adenomas occur in highly localized regions, making them spatially distinct.

- **Primary Error Vector:** The model struggles to differentiate between Gliomas and Meningiomas (Recall for Meningioma dropped to 57%). Both present as large, diffuse masses, confusing the network's boundary detection.
- **Medical Trade-off:** The model prioritizes Recall over Precision for aggressive tumors. False positives are medically acceptable as they trigger secondary screenings, whereas false negatives (missed diagnoses) are fatal.

6 Challenges Faced During Implementation

- **Prediction Output Array Mismatch:** During evaluation, `sklearn.metrics` threw dimensionality errors because the true labels were 1D integers (e.g., `[0, 1, 3]`), but the model outputted 2D Softmax probabilities. We resolved this by implementing `np.argmax(predictions, axis=1)` to collapse the arrays.
- **Catastrophic Forgetting (Model Shock):** During the transition to Phase 2, unfreezing the base model initially caused the accuracy to crash from 65% to 35%. We discovered that the `BatchNormalization` layers were discarding their historical running variances. Programmatically freezing these specific layers stabilized the gradient flow.

7 Instructions to Run Code and Reproduce Results

To reproduce the web application and run real-time inference locally:

1. **Clone the Repository:** Pull the source code from GitHub:

```
git clone https://github.com/Moeeeeed/neuroscan.git
cd neuroscan
```

2. **Environment Setup:** Ensure Python 3.11 is installed. Create a virtual environment and install dependencies:

```
pip install flask tensorflow opencv-python numpy scikit-learn
```

3. **File Structure:** Ensure the trained weights file (`neuroscanModel.keras`) is placed in the root directory alongside the Flask script, and the HTML file is inside a `templates/` folder.
4. **Launch Server:** Execute the main script from your terminal:

```
python F26-23L-0643.py
```

5. **Access Dashboard:** Open a web browser and navigate to `http://127.0.0.1:5001`. Upload any MRI image to observe the model's prediction and confidence score.